

Matching & Mapping Data - Schema

Vu Tuyet Trinh
SoICT - HUST

ONE LOVE. ONE FUTURE.

1

Heterogeneity

- Differences between data sources

- Heterogeneity

- Technology
- Structure
- Meaning
- Governance
- Ownership
- Quality
- Timing

System

storage, APIs, OS, availability

Syntactic

CSV, JSON, XML, Parquet, encodings

Structural

schemas, keys, nesting, granularity

Semantic

meaning, units, codes, context

Quality

missing, duplicates, validity

Temporal

batch vs stream, version

Governance cuts across all categories: access rights, privacy rules, lineage, accountability, and data contracts.

➤ **How to recognize the same real-world object and translate data into the common model**

2

Matching vs. Mapping

- Matching: Which records refer to the same thing?
- Mapping: How should fields and meanings be translated?

Data Matching
identity

Data Mapping
meaning

- Find records, entities, or fields that correspond to the same real-world thing
- Examples: duplicate customers, same product, same supplier, same patient
- Output: match / non-match / possible match, clusters, confidence scores

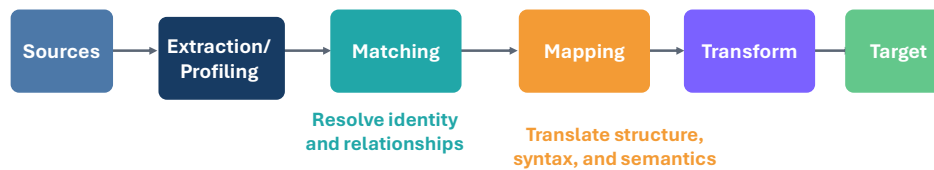
- Specify how source fields and values become target fields and values
- Examples: source.birth_dt → target.date_of_birth; V/M → male/female
- Output: mapping rules, transformations, lineage, validation rules

3

- Explain common matching techniques: rules, similarity functions, blocking, and ML
- • Design schema and semantic mappings between source and target models
- • Evaluate quality with precision, recall, coverage, and error analysis
- • Apply the concepts to a realistic customer integration example

4

Matching & Mapping in Data Integration



5

Data vs Schema (Mapping and Matching)

Schema / metadata level

Schema mapping

Transform attributes and structures into a target schema

Schema matching

Discover which attributes/classes mean the same thing

Mapping: transform / translate

Data mapping

Transform actual values into target representation

Matching: discover correspondence

Data matching

Find duplicate/equivalent records or entities

Data / instance level

6

Data matching vs Schema matching

Aspect	Data matching	Schema matching
Compares	Rows, records, entities, values	Columns, tables, classes, attributes
Main question	Are these records the same entity?	Do these fields/concepts mean the same thing?
Evidence	Names, addresses, IDs, value similarity, behavior	Field names, descriptions, data types, constraints, samples
Output	Matched pairs/clusters; confidence scores	Attribute/table correspondences; match scores
Metrics	Precision, recall, F1, clerical review rate	Precision/recall of correspondences; human validation
Risk	Duplicates remain or false merges occur	Wrong fields get linked, leading to bad transformations

Data mapping vs Schema mapping

Aspect	Data mapping	Schema mapping
Scope	Values and records	Fields, tables, classes, relationships
Question	How should this value be converted?	How should this source structure fill the target structure?
Examples	kg → lb; "VN" → "Vietnam"; split full name	Customer.name → Person.full_name; Order → Transaction
Specification	Rules, functions, lookup tables, cleansing logic	Mapping expressions, joins, projections, constraints
Runtime	Runs on every row/value	Defines the shape of the transformation pipeline
Quality check	Value validity, transformation accuracy, exceptions	Coverage, semantic correctness, referential integrity

String Matching



9

Problems

- Find strings that refer to same real-world entities
 - “David Smith” and “David R. Smith”
 - “1210 W. Dayton St Madison WI” and “1210 West Dayton Madison WI 53706”
- Play critical roles in many DI tasks
 - Schema matching, data matching, information extraction



10

Problem Description

- Two major challenges: accuracy & scalability
- Given two sets of strings X and Y
 - Find all pairs $x \in X$ and $y \in Y$ that refer to the same real-world entity
 - We refer to (x,y) as a match

Set X

x_1 = Dave Smith
 x_2 = Joe Wilson
 x_3 = Dan Smith

(a)

Set Y

y_1 = David D. Smith
 y_2 = Daniel W. Smith

(b)

Matches

(x_1, y_1)
 (x_3, y_2)

(c)



11

11

Techniques

- Similarity measures
 - Sequence-based: edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
 - Set-based: overlap, Jaccard, TF/IDF
 - Hybrid: *generalized Jaccard*, *soft TF/IDF*, *Monge-Elkan*
 - Phonetic: *Soundex*
- Scaling up string matching
 - Inverted index, *size filtering*, *prefix filtering*, *position filtering*, *bound filtering*



TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

12

12

Accuracy Challenges

- Matching strings often appear quite differently
 - Typing and OCR errors: David Smith vs. Davod Smith
 - Different formatting conventions: 10/8 vs. Oct 8
 - Custom abbreviation, shortening, or omission: Daniel Walker Herbert Smith vs. Dan W. Smith
 - Different names, nick names: William Smith vs. Bill Smith
 - Shuffling parts of strings: Dept. of Computer Science, UW-Madison vs. Computer Science Dept., UW-Madison

Accuracy Challenges

- Solution:
 - Use a similarity measure $s(x,y) \in [0,1]$
 - The higher $s(x,y)$, the more likely that x and y match
 - Declare x and y matched if $s(x,y) \geq t$
- Distance measure/cost measure have also been used
 - Same concept
 - But smaller values $\rightarrow$ higher similarities

Scalability Challenges

- Applying $s(x,y)$ to all pairs is impractical
 - Quadratic in size of data
- Solution: apply $s(x,y)$ to only most promising pairs, using a method FindCands
 - For each string $x \in X$
 - use method FindCands to find a candidate set $Z \subseteq Y$
 - for each string $y \in Z$
 - if $s(x,y) \geq t$ then return (x,y) as a matched pair
 - We discuss ways to implement FindCands later

Edit Distance

- Also known as Levenshtein distance
- $d(x,y)$ computes minimal cost of transforming x into y , using a sequence of operators, each with cost 1
 - Delete a character
 - Insert a character
 - Substitute a character with another
- Example: $x = \text{David Smiths}$, $y = \text{Davidd Simth}$, $d(x,y) = 4$, using following sequence
 - Inserting a character d (after David)
 - Substituting m by i
 - Substituting i by m
 - Deleting the last character of x , which is s

Edit Distance

- Models common editing mistakes
 - Inserting an extra character, swapping two characters, etc.
 - So smaller edit distance → higher similarity
- Can be converted into a similarity measure
 - $s(x,y) = 1 - d(x,y) / [\max(\text{length}(x), \text{length}(y))]$
 - Example
 - $s(\text{David Smiths, Davidd Simth}) = 1 - 4 / \max(12, 12) = 0.67$

Computing Edit Distance using Dynamic Programming

- Define $x = x_1x_2 \cdots x_n$, $y = y_1y_2 \cdots y_m$
 - $d(i,j)$ = edit distance between $x_1x_2 \cdots x_i$ and $y_1y_2 \cdots y_j$,
the i -th and j -th prefixes
of x and y

- Recurrence equations

$$d(i,j) = \min \begin{cases} d(i-1,j-1) & \text{if } x_i = y_j \text{ // copy} \\ d(i-1,j-1) + 1 & \text{if } x_i \neq y_j \text{ // substitute} \\ d(i-1,j) + 1 & \text{// delete } x_i \\ d(i,j-1) + 1 & \text{// insert } y_j \end{cases}$$

$$d(i,j) = \min \begin{cases} d(i-1,j-1) + c(x_i, y_j) & \text{// copy or substitute} \\ d(i-1,j) + 1 & \text{// delete } x_i \\ d(i,j-1) + 1 & \text{// insert } y_j \end{cases}$$

$$c(x_i, y_j) = \begin{cases} 0 & \text{if } x_i = y_j, \\ 1 & \text{otherwise} \end{cases}$$

Example

- Cost of dynamic programming is $O(|x||y|)$

■ $x = \text{dva}$, $y = \text{dave}$

	y0	y1	y2	y3	y4
x0		d	a	v	e
x1	0	1	2	3	4
x2	d	1	0	1	
x3	v	2			
x4	a	3			

		y0	y1	y2	y3	y4
			d	a	v	e
x0		0	1	2	3	4
x1	d	1	0	1	2	3
x2	v	2	1	1	1	2
x3	a	3	2	1	2	2

$x = \text{d} - \text{v} \text{ a}$
 $| \quad | \quad |$
 $y = \text{d} \text{ a} \text{ v} \text{ e}$

substitute a with e
 insert a (after d)

Set-based Similarity Measures

- View strings as sets or multi-sets of tokens
- Use set-related properties to compute similarity scores
- Common methods to generate tokens
 - consider words delimited by space
 - possibly stem the words (depending on the application)
 - remove common stop words (e.g., the, and, of)
 - e.g., given "david smith" → generate tokens "david" and "smith"
 - consider q-grams, substrings of length q
 - e.g., "david smith" → the set of 3-grams are {##d, #da, dav, avi, ..., h##}
 - special character # is added to handle the start and end of string

The Overlap Measure

- Let B_x = set of tokens generated for string x
- Let B_y = set of tokens generated for string y
- $O(x,y) = |B_x \cap B_y|$
 - returns the number of common tokens
- E.g., x = dave, y = dav
 - $B_x = \{\#d, da, av, ve, e\# \}$, $B_y = \{\#d, da, av, v\# \}$
 - $O(x,y) = 3$

The TF/IDF Measure

- uses the TF/IDF notion commonly used in IR
 - two strings are similar if they share distinguishing terms
 - e.g., x = Apple Corporation, CA
y = IBM Corporation, CA
z = Apple Corp
 - $s(x,y) > s(x,z)$ using edit distance or Jaccard measure, so x is matched with y → incorrect
 - TF/IDF measure can recognize that Apple is a distinguishing term, whereas Corporation and CA are far more common → correctly match x with z

Term Frequencies and Inverse Document Frequencies

- Assume x and y are taken from a collection of strings
- Each string is converted into a bag of terms called a document
- Define term frequency $tf(t,d)$ = number of times term t appears in document d
- Define inverse document frequency $idf(t) = N / N_d$, number of documents in collection divided by number of documents that contain t
 - note: in practice, $idf(t)$ is often defined as $\log(N / N_d)$, here we will use the above simple formula to define $idf(t)$

23

Example

$$x = aab \Rightarrow B_x = \{a, a, b\}$$

$$y = ac \Rightarrow B_y = \{a, c\}$$

$$z = a \Rightarrow B_z = \{a\}$$

$$tf(a, x) = 2 \quad idf(a) = 3/3 = 1$$

$$tf(b, x) = 1 \quad idf(b) = 3/1 = 3$$

$$\dots \quad idf(c) = 3/1 = 3$$

$$tf(c, z) = 0$$

24

Feature Vectors

- Each document d is converted into a feature vector v_d
- v_d has a feature $v_d(t)$ for each term t
 - value of $v_d(t)$ is a function of TF and IDF scores
 - here we assume $v_d(t) = \text{tf}(t,d) * \text{idf}(t)$

$x = aab \Rightarrow B_x = \{a, a, b\}$

$y = ac \Rightarrow B_y = \{a, c\}$

$z = a \Rightarrow B_z = \{a\}$

$\text{tf}(a, x) = 2 \quad \text{idf}(a) = 3/3 = 1$

$\text{tf}(b, x) = 1 \quad \text{idf}(b) = 3/1 = 3$

... $\text{idf}(c) = 3/1 = 3$

$\text{tf}(c, z) = 0$

	a	b	c
v_x	2	3	0
v_y	1	0	3
v_z	1	0	0

TF/IDF Similarity Score

- Let p and q be two strings, and T be the set of all terms in the collection
- Feature vectors v_p and v_q are vectors in the $|T|$ -dimensional space where each dimension corresponds to a term
- TF/IDF score of p and q is the cosine of the angle between v_p and v_q
 - $s(p,q) = \sum_{t \in T} v_p(t) * v_q(t) / [\sqrt{\sum_{t \in T} v_p(t)^2} * \sqrt{\sum_{t \in T} v_q(t)^2}]$

TF/IDF Similarity Score

- Score is high if strings share many frequent terms
 - terms with high TF scores
- Unless these terms are common in other strings
 - i.e., they have low IDF scores
- Dampening TF and IDF as commonly done in practice
 - use $v_d(t) = \log(\text{tf}(t,d) + 1) * \log(\text{idf}(t))$ instead of $v_d(t) = \text{tf}(t,d) * \text{idf}(t)$
- Normalizing feature vectors

$$v_d(t) = v_d(t) / \sqrt{\sum_{t \in T} v_d(t)^2}$$

Scalability Challenges

- Applying $s(x,y)$ to all pairs is impractical
 - Quadratic in size of data
- Solution: apply $s(x,y)$ to only most promising pairs, using a method FindCands
 - For each string $x \in X$
 - use method FindCands to find a candidate set $Z \subseteq Y$
 - for each string $y \in Z$
 - if $s(x,y) \geq t$ then return (x,y) as a matched pair
 - This is often called a blocking solution
 - Set Z is often called the umbrella set of x
- We now discuss ways to implement FindCands
 - using Jaccard and overlap measures for now

Inverted Index over Strings

- Converts each string $y \in Y$ into a document, builds an inverted index over these documents
- Given term t , use the index to quickly find documents of Y that contain t

29

Example

Set X

- 1: {lake, mendota}
- 2: {lake, monona, area}
- 3: {lake, mendota, monona, dane}

Set Y

- 4: {lake, monona, university}
- 5: {monona, research, area}
- 6: {lake, mendota, monona, area}

(a)

Terms in Y	ID Lists
area	5,6
lake	4, 6
mendota	6
monona	4, 5, 6
research	5
university	4

(b)

30

Limitations

- The inverted list of some terms (e.g., stop words) can be very long → costly to build and manipulate such lists
- Requires enumerating all pairs of strings that share at least one term. This set can still be very large in practice.

31

Data Matching

32

Introduction

- Data matching: find structured **data items that refer to the same real-world entity**
 - entities may be represented by tuples, XML elements, or RDF triples, not by strings as in string matching
 - e.g., (David Smith, 608-245-4367, Madison WI) vs (D. M. Smith, 245-4367, Madison WI)
- Data matching arises in many integration scenarios
 - **merging** multiple databases with the **same schema**
 - **joining** rows from sources with **different schemas**
 - matching a user query to a data item

Problem Definition

- Given two relational tables X and Y with identical schemas
 - assume each tuple in X and Y describes an entity (e.g., person)
- We say tuple $x \in X$ matches tuple $y \in Y$ if they refer to the same real-world entity
 - (x,y) is called a match
- **Goal: find all matches between X and Y**

Data matching techniques

- Rule-based matching
- Learning- based matching
- Matching by clustering
- Probabilistic approaches to matching
- Collective matching
- Scaling up data matching

35

Example

Table X					Table Y					Matches	
	Name	Phone	City	State		Name	Phone	City	State		
X_1	Dave Smith	(608) 395 9462	Madison	WI	Y_1	David D. Smith	395 9426	Madison	WI		(X_1, Y_1)
X_2	Joe Wilson	(408) 123 4265	San Jose	CA	Y_2	Daniel W. Smith	256 1212	Madison	WI		(X_2, Y_2)
X_3	Dan Smith	(608) 256 1212	Middleton	WI							

(a)

(b)

(c)

- Other variations
 - Tables X and Y have different schemas
 - Match tuples within a single table X
 - The data is not relational, but XML or RDF

36

Why is This Different than String Matching?

- In theory, can treat each tuple as a string by concatenating the fields, then apply string matching techniques
- But doing so makes it hard to apply sophisticated techniques and domain-specific knowledge
- E.g., consider matching tuples that describe persons
 - suppose we know that in this domain two tuples match if the names and phone match exactly
 - this knowledge is hard to encode if we use string matching
 - so it is better to keep the fields apart

Challenges

- Same as in string matching
- How to match accurately?
 - difficult due to variations in formatting conventions, use of abbreviations, shortening, different naming conventions, omissions, nicknames, and errors in data
 - several common approaches: rule-based, learning-based, clustering, probabilistic, collective
- How to scale up to large data sets
 - again many approaches have been developed, as we will discuss

Techniques

- Rule-based matching
- Matching by clustering
- *Learning- based matching*
- *Probabilistic approaches to matching*
- *Collective matching*
- Scaling up data matching

Rule-based Matching

- The developer writes rules that specify when two tuples match
 - typically after examining many matching and non-matching tuple pairs, using a development set of tuple pairs
 - rules are then tested and refined, using the same development set or a test set
- Many types of rules exist, we will consider
 - linearly weighted combination of individual similarity scores
 - logistic regression combination
 - more complex rules

Linearly Weighted Combination Rules

- Compute the sim score between tuples x and y as a linearly weighted combination of individual sim scores
 - $\text{sim}(x,y) = \sum_{i=1}^n \alpha_i * \text{sim}_i(x,y)$
 - n is number of attributes in each table
 - $s_i(x,y)$ is a sim score between the i -th attributes of x and y
 - $\alpha_i \in [0,1]$ is a pre-specified weight that indicates the important of the i -th attribute to $\text{sim}(x,y)$, such that $\sum_{i=1}^n \alpha_i = 1$
- We declare x and y matched if $\text{sim}(x,y) \geq \beta$ for a pre-specified β , and not matched otherwise
 - in another variation: declare x and y matched if $\text{sim}(x,y) \geq \beta$, not matched if $\text{sim}(x,y) < \gamma$. and subject to human review

41

Example

- $\text{sim}(x,y) = 0.3s_{\text{name}}(x,y) + 0.3s_{\text{phone}}(x,y) + 0.1s_{\text{city}}(x,y) + 0.3s_{\text{state}}(x,y)$
 - $s_{\text{name}}(x,y)$: based on Jaro-Winkler
 - $s_{\text{phone}}(x,y)$: based on edit distance between x 's phone (after removing area code) and y 's phone
 - $s_{\text{city}}(x,y)$: based on edit distance
 - $s_{\text{state}}(x,y)$: based on exact match; yes = 1, no = 0

Table X

	Name	Phone	City	State
x_1	Dave Smith	(608) 395 9462	Madison	WI
x_2	Joe Wilson	(408) 123 4265	San Jose	CA
x_3	Dan Smith	(608) 256 1212	Middleton	WI

(a)

Table Y

	Name	Phone	City	State
y_1	David D. Smith	395 9426	Madison	WI
y_2	Daniel W. Smith	256 1212	Madison	WI

(b)

Matches
(x_1, y_1)
(x_3, y_2)

(c)

42

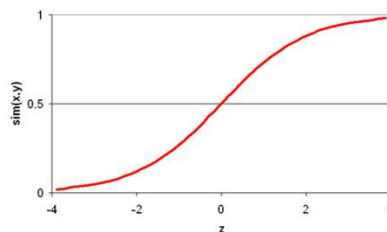
Pros and Cons

- Pros
 - conceptually simple, easy to implement
 - can learn weights α_i from training data
- Cons
 - an increase $\pm$ in the value of any s_i will cause a linear increase $\alpha_i * \pm$ in the value of s
 - in certain scenarios this is not desirable, there after a certain threshold an increase in s_i should count less (i.e., “diminishing returns” should kick in)
 - e.g., if $s_{\text{name}}(x,y)$ is already 0.95 then the two names already very closely match
 - so any increase in $s_{\text{name}}(x,y)$ should contribute only minimally

43

Logistic Regression Rules

- address the diminishing-returns problem
- $\text{sim}(x,y) = 1 / (1 + e^{-z})$, where $z = \sum_{i=1}^n \alpha_i * \text{sim}_i(x,y)$
 - here α_i are not constrained to be in $[0,1]$ and sum to 1
 - so z goes from $-\infty$ to $+\infty$, in which case $\text{sim}(x,y)$ gradually increases, but minimally so after z has exceeded a certain value
→ ensuring diminishing returns



44

Logistic Regression Rules

- Are also very useful in situations where
 - there are many “signals” (e.g., 10-20) that can contribute to whether two tuples match
 - we don’t need all of these signals to “fire” in order to conclude that the tuples match
 - as long as a reasonable number of them fire, we have sufficient confidence
- Logistic regression is a natural fit for such cases
- Hence is quite popular as a first matching method to try

More Complex Rules

- Appropriate when we want to encode more complex matching knowledge
 - e.g., two persons match if names match approximately and either phones match exactly or addresses match exactly
 1. If $s_{\text{name}}(x,y) < 0.8$ then return “not matched”
 2. Otherwise if $e_{\text{phone}}(x,y) = \text{true}$ then return “matched”
 3. Otherwise if $e_{\text{city}}(x,y) = \text{true}$ and $e_{\text{state}}(x,y) = \text{true}$ then return “matched”
 4. Otherwise return “not matched”

Pros and Cons of Rule-Based Approaches

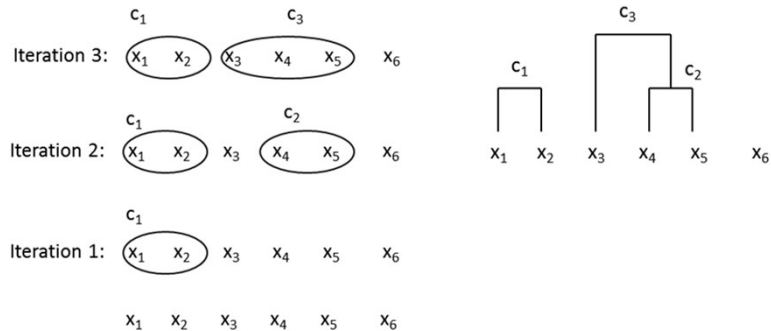
- Pros
 - easy to start, conceptually relatively easy to understand, implement, debug
 - typically run fast
 - can encode complex matching knowledge
- Cons
 - can be labor intensive, it takes a lot of time to write good rules
 - can be difficult to set appropriate weights
 - in certain cases it is not even clear how to write rules
 - learning-based approaches address these issues

Matching by Clustering

- Many common clustering techniques have been used
 - agglomerative hierarchical clustering (AHC), k-means, graph-theoretic, ...
 - here we focus on AHC, a simple yet very commonly used one
- AHC
 - partitions a given set of tuples D into a set of clusters
 - all tuples in a cluster refer to the same real-world entity, tuples in different clusters refer to different entities
 - begins by putting each tuple in D into a single cluster
 - iteratively merges the two most similar clusters
 - stops when a desired number of clusters has been reached, or until the similarity between two closest clusters falls below a pre-specified threshold

Example

- $\text{sim}(x,y) = 0.3s_{\text{name}}(x,y) + 0.3s_{\text{phone}}(x,y) + 0.1s_{\text{city}}(x,y) + 0.3s_{\text{state}}(x,y)$



Computing a Similarity Score between Two Clusters

- Let c and d be two clusters
- Single link: $s(c,d) = \min_{x_i \in c, y_j \in d} \text{sim}(x_i, y_j)$
- Complete link: $s(c,d) = \max_{x_i \in c, y_j \in d} \text{sim}(x_i, y_j)$
- Average link: $s(c,d) = \frac{[\sum_{x_i \in c, y_j \in d} \text{sim}(x_i, y_j)]}{[\# \text{ of } (x_i, y_j) \text{ pairs}]}$
- Canonical tuple
 - create a canonical tuple that represents each cluster
 - sim between c and d is the sim between their canonical tuples
 - canonical tuple is created from attribute values of the tuples
 - e.g., "Mike Williams" and "M. J. Williams" => "Mike J. Williams"
 - (425) 247 4893 and 247 4893 => (425) 247 4893

Key Ideas underlying the Clustering Approach

- View matching tuples as the problem of constructing entities (i.e., clusters)
- The process is iterative
 - leverage what we have known so far to build “better” entities
- In each iteration merge all matching tuples within a cluster to build an “entity profile”, then use it to match other tuples => **merging then exploiting the merged information to help matching**
- These same ideas appear in subsequent approaches that we will cover

Techniques

- Rule-based matching
- Matching by clustering
- *Learning- based matching*
- *Probabilistic approaches to matching*
- *Collective matching*
- Scaling up data matching

Scaling up Rule-based Matching

- Two goals: minimize # of tuple pairs to be matched and minimize time it takes to match each pair
- For the first goal:
 - hashing
 - sorting
 - indexing
 - canopies
 - using representatives
 - combining the techniques
- Hashing
 - hash tuples into buckets, match only tuples within each bucket
 - e.g., hash house listings by zipcode, then match within each zip

Scaling up Rule-based Matching

- Sorting
 - use a key to sort tuples, then scan the sorted list and match each tuple with only the previous (w-1) tuples, where w is a pre-specified window size
 - key should be strongly “discriminative”: brings together tuples that are likely to match, and pushes apart tuples that are not. Example keys: soc sec, student ID, last name, soundex value of last name
 - employs a stronger heuristic than hashing: also requires that tuples likely to match be within a window of size w
 - but is often faster than hashing because it would match fewer pairs

Scaling up Rule-based Matching

- Indexing
 - index tuples such that given any tuple a , can use the index to quickly locate a relatively small set of tuples that are likely to match a
 - e.g., inverted index on names
- Canopies
 - use a computationally cheap sim measure to quickly group tuples into overlapping clusters called canopies (or umbrella sets)
 - use a different (far more expensive) sim measure to match tuples within each canopy
 - e.g., use TF/IDF to create canopies

Scaling up Rule-based Matching

- Using representatives
 - applied during the matching process
 - assigns tuples that have been matched into groups such that those within a group match and those across groups do not
 - create a representative for each group by selecting a tuple in the group or by merging tuples in the group
 - when considering a new tuple, only match it with the representatives
- Combining the techniques
 - e.g., hash houses into buckets using zip codes, then sort houses within each bucket using street names, then match them using a sliding window

Scaling up Rule-based Matching

- For the second goal of minimizing time it takes to match each pair
 - no well-established technique as yet
 - tailor depending on the application and the matching approach
 - e.g., if using a simple rule-based approach that matches individual attributes then combines their scores using weights
 - can use short circuiting: stop the computation of the sim score if it is already so high that the tuple pair will match even if the remaining attributes do not match

Scaling up Other Matching Methods

- Learning, clustering, probabilistic, and collective approaches often face similar scalability challenges, and can benefit from the same solutions
- Probabilistic approaches raise additional challenges
 - if model has too many parameters => difficult to learn efficiently, need a large # of training data to learn accurately
 - make independence assumptions to reduce # of parameters
- Once learned, inference with model is also time costly
 - use approximate inference algorithms
 - simplify model so that closed form equations exist
- EM algorithm can be expensive
 - truncate EM, or initializing it as accurately as possible

Scaling up Using Parallel Processing

- Commonly done in practice
- Examples
 - hash tuples into buckets, then match each bucket in parallel
 - match tuples against a taxonomy of entities (e.g., a product or Wikipedia-like concept taxonomy) in parallel
 - two tuples are declared matched if they match into the same taxonomic node
 - a variant of using representatives to scale up, discussed earlier

Summary

- Critical problem in data integration
- Huge amount of work in academia and industry
 - Rule-based matching
 - Learning- based matching
 - Matching by clustering
 - Probabilistic approaches to matching
 - Collective matching